

PART I — FOUNDATIONS

Chapter 2

Anatomy of a Retrieval-Augmented Generation System

“A complicated system that works is invariably found to have evolved from a simple system that worked.”
— John Gall

Chapter 2 — Anatomy of a Retrieval-Augmented Generation System

Chapter 1 made the case for why RAG exists. This chapter opens the box and shows you what is inside. By the end of it, you will have a clear, working mental model of every moving part — and you will have built a tiny end-to-end RAG system in around forty lines of Python that actually answers questions over your own documents.

The structure of this chapter mirrors how RAG systems are actually built in practice. We will start with the two-pipeline split that nearly every RAG system follows. Then we will trace a full question through the end-to-end flow, naming each component as it does its job. Then we will look at all six core components side-by-side, so you know exactly what each one is responsible for and what its inputs and outputs are. Then comes the minimal code — small enough to read in one sitting, complete enough to actually run. And finally, before we move on, we will be honest about what this minimal pipeline cannot do — which is precisely the gap that the rest of the book exists to fill.

Treat this chapter as the architectural skeleton of everything that follows. Every later chapter takes one of these components and goes deep. Chunking strategies, embedding models, vector databases, retrievers, prompts, agents, self-learning — they are all elaborations on the simple anatomy you are about to learn.

2.1 The two pipelines: ingestion and retrieval

Almost every RAG system, no matter how advanced, decomposes into two pipelines that run at different times and serve different purposes. Confusing them is the most common architectural mistake beginners make. Let us pin them down before any code appears.

Definition — Ingestion pipeline

The offline pipeline that takes your raw documents (PDFs, text files, web pages, spreadsheets, databases) and turns them into searchable, embedded chunks stored in a vector database. It runs once, and again whenever your knowledge base changes. The user is not in the loop.

Definition — Retrieval pipeline

The online pipeline that runs every time a user asks a question. It takes the query, finds the most relevant chunks from the vector database, combines them with the original question into an augmented prompt, sends that prompt to an LLM, and returns the grounded answer. The user is very much in the loop, and latency matters.

These two pipelines do not run together. The ingestion pipeline runs in batch — typically once a day, once a week, or whenever new documents arrive. The retrieval pipeline runs in milliseconds, every single time a user sends a query. The first prepares the kitchen; the second cooks each meal to order.

Analogy — The library cataloguer and the librarian

Imagine a public library. Long before any reader walks in, a *cataloguer* has already shelved every book, written its index card, and filed it under the right Dewey decimal code. That is the ingestion pipeline — slow, deliberate, run once per acquisition. Then a reader walks up and asks, “Do you have anything on Byzantine mosaics?” The *librarian* — this is the retrieval pipeline — uses the prepared index to fetch the three most relevant books in seconds. The cataloguer is paid by the hour to be thorough; the librarian is judged on how fast and accurate the answer is. Same library, two completely different jobs.

Why does this split exist? Because embedding documents is expensive and slow, and we do not want to redo it on every query. By separating the work, we pay the heavy cost once (at ingestion) and the light cost repeatedly (at retrieval). This is also why the vector database sits squarely between the two pipelines — it is the durable, persisted artifact that the ingestion pipeline produces, and the retrieval pipeline consumes.

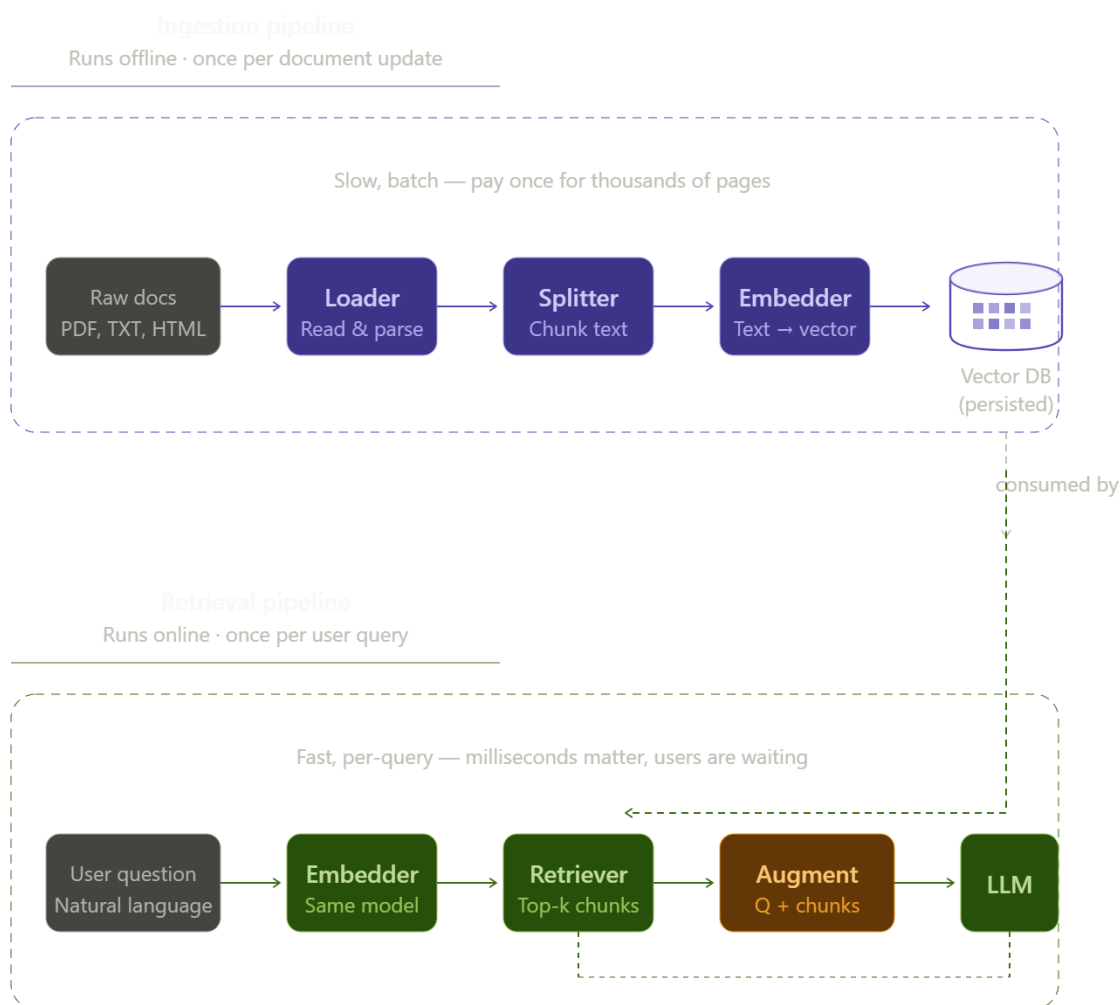


Figure 2.1 — The two pipelines of a RAG system. The ingestion pipeline runs once per document update and produces a persistent vector database. The retrieval pipeline runs once per user question and consumes that database to produce grounded answers.

Note one subtle but important detail: the embedder appears in both pipelines. During ingestion, it embeds documents; during retrieval, it embeds the user's query. It must be the same model in both cases — a query embedded by model A cannot be meaningfully compared to documents embedded by model B, because they live in different vector spaces. We will return to this point in Chapter 7 on embeddings.

2.2 End-to-end flow diagram

Let us now follow a single concrete question all the way through both pipelines. Suppose your company has uploaded a stack of HR policy PDFs into a folder, you have run the ingestion pipeline once, and an employee now types into the chat interface:

Worked example — the question

“What is our company's parental leave policy?”

Here is what happens, end to end, in a working RAG system. Read this slowly — every step here is an entire chapter later in the book.

Step 1 — Embed the query. The user's question is passed through the same embedding model that was used during ingestion (for example, all-MiniLM-L6-v2). The model converts the 12-word string into a 384-dimensional vector. This vector is the query's “address” in semantic space.

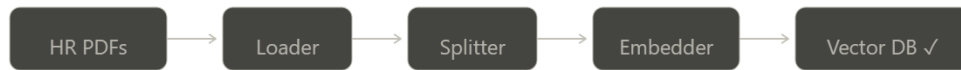
Step 2 — Similarity search. The retriever asks the vector database: “Find me the top-k chunks whose embeddings are closest to this query vector.” The database, which already contains thousands of pre-embedded chunks from the HR PDFs, returns (typically) the 3-5 most similar chunks along with their similarity scores and source metadata.

Step 3 — Assemble the augmented prompt. The retrieved chunks are concatenated and inserted into a prompt template alongside the user's original question. The result is a single, larger prompt that contains both the question and the evidence the model is expected to use.

Step 4 — Generate the answer. The augmented prompt is sent to an LLM (such as Llama 3.3 70B via Groq, GPT-4o via OpenAI, or a local model via Ollama). The model is instructed to answer the question using only the provided context. It returns a grounded answer with optional source citations.

Step 5 — Return to the user. The application surfaces the answer along with the source documents and similarity scores, so the user can verify the claim if they want to. End of the pipeline.

Step 0 — Ingestion (already done, persisted on disk)



Steps 1–5 — Retrieval (runs now, on every query)

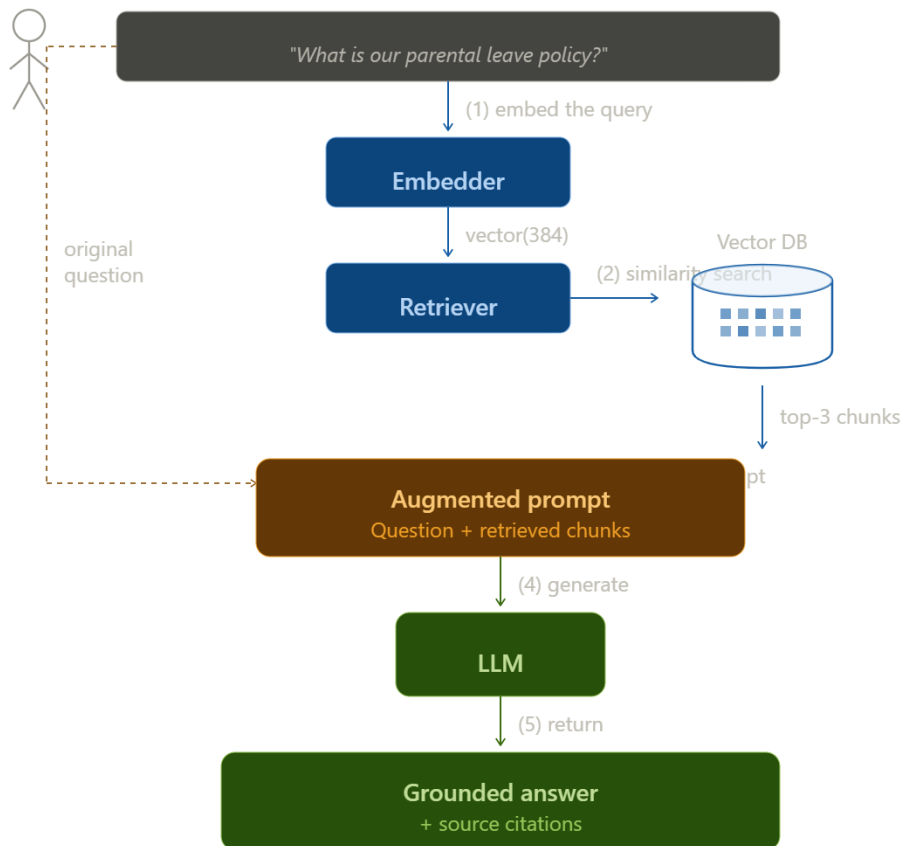


Figure 2.2 — One question, five steps. Steps 1–5 happen on every query in tens to hundreds of milliseconds; step 0 already completed offline.

If you internalize this five-step flow, you understand classical RAG. Everything else — chunking strategies, hybrid search, re-rankers, agentic loops, self-learning — is a refinement, replacement, or extension of one or more of these five steps. We will spend the next thirty chapters making each step better. But the skeleton you see in Figure 2.2 will not change.

2.3 Core components at a glance

RAG systems are built from six core components. Each one has a single, well-defined responsibility. Read this section like a glossary: the names, definitions, and contracts here will be referenced constantly throughout the rest of the book, so it is worth pinning them down precisely.

Component	Responsibility	Input	Output
Loader	Read raw files from disk, URL, or database and produce structured Document objects	File paths or URLs	Document objects with <code>page_content</code> and <code>metadata</code>
Splitter	Break long documents into smaller, overlapping chunks suitable for embedding	Document objects	Smaller Document chunks (typically 500–1500 chars)
Embedder	Convert text into a fixed-size dense vector that represents its meaning	A string of text	A numerical vector (e.g., 384 floats)
Vector DB	Store embeddings + metadata and answer nearest-neighbor queries efficiently	Vectors + metadata to insert; query vectors to search	Top-k matching chunks with scores
Retriever	Orchestrate query embedding and vector search, return a clean list of chunks	User query string	Ranked list of relevant chunks with scores and sources
Generator	Combine question + retrieved chunks into a prompt and call an LLM	Question + retrieved chunks	Final natural-language answer

Table 2.1 — The six core components of a RAG system, their responsibilities, and their input/output contracts.

A few clarifications about how these fit together in practice.

Loader

The loader's job is purely mechanical: open a file, parse its bytes, and produce a unified Document object regardless of the source format. The same loader interface should handle a PDF, a Word document, a Markdown file, an HTML page, and a CSV row, so that downstream components do not need to care where the text came from. In our codebase, `UnstructuredLoader` handles most cases; `PyPDFLoader` and `PyMuPDFLoader` are specialized PDF loaders we will compare in Chapter 5.

Splitter

Splitters exist because LLMs and embedding models have finite context windows, and because retrieval works much better on focused chunks than on entire documents. The default workhorse is `RecursiveCharacterTextSplitter`, which tries to split on natural boundaries (paragraphs first, then sentences, then words) and falls back to fixed-size cuts only when necessary. Two parameters dominate: `chunk_size` (how big each chunk is, typically 1000 characters) and `chunk_overlap` (how much adjacent chunks share, typically 200 characters, to preserve context across boundaries). We will tune these in Chapter 7.

Embedder

The embedder is a pretrained neural network that maps text to a vector. The vector's dimensionality is fixed by the model — 384 for all-MiniLM-L6-v2, 768 for many BERT-based models, 1536 or 3072 for OpenAI's text-embedding-3 family. The model has been trained so that semantically similar texts produce vectors that are close together in this space, while unrelated texts produce vectors that are far apart. This is the mathematical foundation of every retrieval step we will write.

Vector database

A vector database is not a relational database with extra columns — it is purpose-built for one operation: given a query vector, return the *k* nearest stored vectors as fast as possible. Modern vector DBs use approximate-nearest-neighbor algorithms (HNSW, IVF, ScaNN) to do this in milliseconds across millions of vectors. We will use ChromaDB throughout this book because it is local, free, and excellent for development. Chapter 8 surveys alternatives — FAISS, Pinecone, Weaviate, Qdrant, Milvus — and explains when to reach for which.

Retriever

The retriever is a thin orchestration layer that sits on top of the embedder and the vector database. Its job is to take a raw user query, embed it, search the vector store, optionally apply filters or rerank, and return a clean list of chunks with scores and metadata. In simple systems it is fewer than 30 lines of code. In more advanced systems it grows to combine dense and sparse search, apply cross-encoder reranking, and merge results from multiple collections — all of which we will build in Chapters 11 and 12.

Generator

The generator is the LLM call wrapped in a prompt template. It takes the user's question and the retrieved chunks, fits them into a template like “Answer the following question using only the provided context: ...”, sends that to the model, and returns the response. The generator is where prompt engineering lives, where citations are formatted, and where the model is instructed to refuse to invent answers when the context does not support them. Chapters 13 and 14 are dedicated to it.

Mental model — six Lego bricks

Think of these six components as Lego bricks with standard connectors. Every RAG system you will ever encounter — toy demo, production app, research paper, agentic system — is a different arrangement of these same bricks, sometimes with extras (rerankers, query rewriters, evaluators) snapped on. Once you can name the bricks, every system you read about becomes legible.

2.4 A minimal RAG in ~40 lines of code

Enough theory. Here is a complete, runnable RAG system in roughly forty lines of Python. It uses LangChain for the loader and splitter, sentence-transformers for the embedder, ChromaDB as the vector database, and Groq's Llama 3.3 70B as the generator. Read it once for shape, then we will walk through it.

Before running it, you would need a .env file with GROQ_API_KEY=... and a docs/ folder containing at least one .txt or .pdf file. Installation: `pip install langchain langchain-community langchain-groq chromadb sentence-transformers python-dotenv unstructured`.

```
# minimal_rag.py — a working RAG in ~40 lines
import os
from dotenv import load_dotenv
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from sentence_transformers import SentenceTransformer
import chromadb
from langchain_groq import ChatGroq

load_dotenv()

# 1. LOAD — read every .txt under ./docs into Document objects
loader = DirectoryLoader("./docs", glob="**/*.txt", loader_cls=TextLoader)
documents = loader.load()

# 2. SPLIT — break each document into ~1000-char chunks with 200-char overlap
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(documents)

# 3. EMBED — convert each chunk into a 384-dim vector
embedder = SentenceTransformer("all-MiniLM-L6-v2")
texts = [c.page_content for c in chunks]
vectors = embedder.encode(texts).tolist()

# 4. STORE — persist vectors + text + metadata into ChromaDB
client = chromadb.PersistentClient(path="./chroma_store")
collection = client.get_or_create_collection("my_docs")
collection.add(
    ids=[f"chunk_{i}" for i in range(len(chunks))],
    embeddings=vectors,
    documents=texts,
    metadatas=[c.metadata for c in chunks],
)
```

```
# 5. ASK — embed the query, retrieve top-3, build prompt, call LLM
llm = ChatGroq(model_name="llama-3.3-70b-versatile", temperature=0.1)
```

```
def ask(question: str) -> str:
    q_vec = embedder.encode(question).tolist()
    hits = collection.query(query_embeddings=[q_vec], n_results=3)
    context = "\n\n".join(hits["documents"][0])
    prompt = (
        "Answer the question using only the context below. "
        "If the context does not contain the answer, say so.\n\n"
        f"Context:\n{context}\n\nQuestion: {question}"
    )
    return llm.invoke(prompt).content

if __name__ == "__main__":
    print(ask("What is our parental leave policy?"))
```

That is the entire RAG system. About forty lines once you strip the comments. Let us walk through what each section does, mapping back to Section 2.3:

- **Section 1 (lines under # 1. LOAD)** is the loader. DirectoryLoader walks the docs/ folder and produces a list of Document objects, each with page_content and metadata fields.
- **Section 2 (# 2. SPLIT)** is the splitter. RecursiveCharacterTextSplitter breaks each document into overlapping ~1000-character chunks.
- **Section 3 (# 3. EMBED)** is the embedder. SentenceTransformer converts every chunk into a 384-dimensional vector in one batch call.
- **Section 4 (# 4. STORE)** is the vector database. ChromaDB persists the vectors, the original text, and the metadata together, indexed for fast nearest-neighbor search.
- **Section 5 (# 5. ASK)** combines the retriever and the generator. ask() embeds the question, queries the collection for the top 3 matches, builds an augmented prompt, and sends it to the LLM.

This script does something genuinely useful. Drop a few company policy documents into docs/, run it once, and you have a question-answering system that grounds its answers in your specific documents — not in whatever the LLM happened to read during training. Hallucination has dropped sharply because the model is now reading actual evidence on every call. Knowledge cutoff is gone, because adding new documents is as simple as dropping them in the folder and rerunning the script.

Note — ingestion vs retrieval in this script

In this minimal version, ingestion (sections 1–4) and retrieval (section 5) live in the same file and run in the same process. In any real system you would split them: a separate ingest.py runs sections 1–4 once, persists to ChromaDB, and exits; a separate query.py loads ChromaDB and only runs section 5 per question. We will do exactly that split starting in Chapter 9.

Stop here, run this script on your own documents, and convince yourself it works. Everything in the next thirty chapters is a refinement of these forty lines.

2.5 Limits of "traditional" RAG — the case for something smarter

The forty-line system you just saw is what the literature calls traditional RAG, naive RAG, or classical RAG. It is genuinely useful, and for many simple use cases it is enough. But it has clear, predictable failure modes — and recognizing them is what motivates the rest of this book.

Definition — Traditional (or naive) RAG

A RAG system whose retrieval pipeline runs as a single fixed pass: embed the query exactly once, retrieve top-k once, build one prompt, generate one answer. The pipeline has no awareness of whether the retrieval succeeded, no ability to retry with a different query, and no memory of past queries or feedback.

Here are the concrete failure modes that traditional RAG cannot escape, with the chapter numbers where we fix each one.

Limit 1 — One shot, no retries

If the user's first phrasing of the question retrieves poor chunks, the pipeline answers anyway, often badly. Real users phrase questions in ways that do not match how the documents are written. Ask “How many vacation days do I get?” of a corpus that uses the phrase “annual leave entitlement” and a single similarity search may miss entirely. Traditional RAG has no mechanism to notice the mismatch, no mechanism to rephrase the query, and no mechanism to try again. (Fixed in Part IV — agentic RAG.)

Limit 2 — No notion of "is this answer good?"

After generating, traditional RAG simply returns. It does not check whether the answer is grounded in the retrieved chunks, whether it actually answered the user's question, or whether the model hedged because it had no real evidence. The user is the only quality check. (Fixed in Chapter 16 with `check_answer_quality` and in Chapter 25 with user thumbdowns.)

Limit 3 — No memory across questions

Each query is independent. If you have already asked five questions about parental leave and the system has retrieved excellent chunks each time, the sixth question starts from zero. There is no learned shortcut, no cached context, no awareness that this user is in a particular line of inquiry. (Fixed in Chapter 16 with short-term session memory and Chapter 27 with hybrid retrieval over learned QA.)

Limit 4 — No learning from feedback

If a user thumbs-down an answer, traditional RAG has nowhere to put that signal. Tomorrow's identical query produces an identical bad answer. The system never improves with use — it has the same blind spots on day 365 as on day 1. (Fixed in Part V — Chapters 22 through 27 build the entire self-learning loop.)

Limit 5 — Aggregation and computation impossible

Ask “What is the average employee tenure across the HR records?” and traditional RAG fails completely. Similarity search retrieves rows that mention tenure, but cannot compute averages. RAG is an information-retrieval tool, not a calculator. (Fixed in Chapter 6 with the two-track architecture — RAG for prose, pandas for structured data.)

Limit 6 — Retrieval quality is not adaptive

The same top-k, same chunk size, and same embedding model are used for every query — whether the user is asking a one-word factoid lookup or a multi-part comparison. Traditional RAG cannot decide that a particular query needs a wider net or a finer-grained one. (Addressed in Chapter 12 with hybrid search, MMR, multi-query retrieval, and HyDE.)

Notice the pattern. Every limitation comes from the same root cause: the pipeline is fixed. There is no decision-maker. The path from question to answer is a hard-coded sequence of five steps, and at no point does the system ask itself, “Is this working? Should I try something different?”

Key insight — what “agentic” really means

Agentic RAG, the topic of Part IV, is what you get when you replace the fixed pipeline with a *decision-making loop*. An LLM, given access to retrieval as a callable tool, decides on its own *when* to retrieve, *what* to retrieve, *where* to retrieve from, and *how many times* to retrieve before answering. The pipeline becomes a strategy, not a script. Layered on top of that, *self-learning* is what you get when the system also remembers which strategies worked, which ones failed, and feeds those lessons back into future queries.

The roadmap from here

With the anatomy now clear, the rest of the book is a guided tour of upgrades to this skeleton. Part II (Chapters 4-9) deepens the ingestion pipeline — better loaders, smarter chunking, richer metadata, more capable embedders, and a proper persistent store. Part III (Chapters 10-14) deepens the retrieval pipeline — hybrid search, reranking, MMR, multi-query, HyDE, and grounded prompt design. Part IV (Chapters 15-21) replaces the fixed pipeline with an agentic loop, with tool use, iteration caps, and quality checks. Part V (Chapters 22-27) adds the self-learning layer — feedback capture, failed-variant blocklists, user-thumbdown memory, and synthetic Q&A distillation. Part VI (Chapters 28-32) covers production concerns — cost, security, deployment, and how to talk about the system in an interview.

By the end of Chapter 32, the forty-line script you saw in Section 2.4 will have grown into a self-learning agentic RAG system that decides for itself when to retrieve, evaluates its own answers, remembers what worked, and gets better with use. The skeleton stays the same. The intelligence around it is what we are about to build.

Chapter summary

A RAG system is two pipelines connected by a vector database. The ingestion pipeline (loader → splitter → embedder → vector DB) runs offline and prepares your documents for search. The retrieval pipeline (embedder → retriever → augmented prompt → generator) runs per query and turns each user question into a grounded answer. Six core components — loader, splitter, embedder, vector DB, retriever, generator — make up nearly every RAG system, and each has a clean, well-defined contract. A working RAG system fits in roughly forty lines of code. But that classical, fixed-pipeline form has predictable limits — no retries, no quality checks, no memory, no learning, no aggregation, no adaptive retrieval — which together motivate the agentic and self-learning extensions that fill the rest of this book.

In the next chapter, we will set up your development environment, install the exact stack we will use throughout the book, and run the smoke test that confirms everything works. Then, starting in Chapter 4, we begin building each component properly — starting with the most fundamental abstraction of them all: the Document object.